📚 جزوه سیستم عامل | فصل ۸: بن‌بست (Deadlock)

📝 بخش ۷ و ۸: تشخیص بن‌بست و روش‌های ترمیم

🔍 بخش ۷: تشخیص بن‌بست (Deadlock Detection)

اگر سیستم از روش‌های پیشگیری یا اجتناب (که در بخش‌های قبل گفتیم) استفاده نکند، بن‌بست ممکن است رخ دهد. در این صورت، سیستم‌عامل باید الگوریتمی داشته باشد که:

1. وضعیت سیستم را بررسی کند تا ببیند آیا بن‌بست رخ داده؟
2. اگر رخ داده، چه کسانی درگیر هستند؟

روش تشخیص بستگی به نوع منابع دارد:

الف) اگر هر منبع فقط ۱ نمونه داشته باشد:

✅ **استفاده از "گراف انتظار کشی" (Wait-for Graph)**

این گراف، نسخه ساده‌شده‌ی همان گراف تخصیص منبع است.

- **روش ساخت:** گره‌های "منبع" را حذف می‌کنیم و یال‌ها را مستقیم بین نخ‌ها می‌کشیم.
- **معنی یال $T_i \rightarrow T_j$:** یعنی نخ i منتظر نخی است که دست نخ j است.
- **قانون طلایی:** در گراف انتظار کشی، اگر **دور (Cycle)** وجود داشته باشد $\leftarrow\$ **قطعاً بن‌بست داریم.**
- **پیچیدگی:** برای پیدا کردن دور در گراف، الگوریتم از مرتبه $O(n^2)$ زمان می‌برد.

ب) اگر منابع دارای چند نمونه باشند:

✅ **استفاده از الگوریتمی شبیه بانکدار**

چون گراف در اینجا جواب نمی‌دهد (ممکن است دور باشد ولی بن‌بست نباشد)، از ماتریس‌ها استفاده می‌کنیم.

- **تفاوت با الگوریتم اجتناب:** در اینجا ماتریس Max نداریم! به جایش ماتریس **Request** (درخواست‌های فعلی) داریم.
- **منطق الگوریتم:**
  1. دنبال نخی می‌گردیم که درخواست‌های فعلی‌اش (Request) کمتر یا مساوی موجودی آزاد (Available) باشد.
  2. فرض می‌کنیم این نخ منابعش را می‌گیرد، کارش تمام می‌شود و منابعی که دستش بود (Allocation) را پس می‌دهد.
  3. این کار را تکرار می‌کنیم.
  4. اگر در آخر، نخی باقی ماند که نتوانستیم کارش را تمام کنیم (یعنی Finish == false ماند)، آن نخ در **بن‌بست** است.

⏱️ کی الگوریتم تشخیص را اجرا کنیم؟

اجرای این الگوریتم‌ها سربار (هزینه) دارد.

1. **هر بار که یک درخواست رد می‌شود؟** (خیلی دقیق ولی **گران** و کند کننده سیستم).
2. **به صورت دوره‌ای؟** (مثلاً هر یک ساعت یا وقتی مصرف CPU زیر ۴۰٪ آمد). این روش بهینه‌تر است.

🛠️ بخش ۸: ترمیم و رفع بن‌بست (Recovery)

وقتی بن‌بست را تشخیص دادیم، باید آن را بشکنیم. دو راه اصلی داریم:

روش ۱: پایان دادن به پردازه‌ها (Process Termination) 🔪

می‌توانیم نخ‌ها را بکشیم (Abort کنیم) تا منابع آزاد شوند.

- **حالت افراطی: همه** پردازه‌های درگیر بن‌بست را بکشیم!
  - *مزیت:* بن‌بست قطعاً می‌شکند.
  - *عیب:* تمام کارهایی که تا الان کرده بودند هدر می‌رود (مثلاً محاسبه‌ای که ۱۰ ساعت طول کشیده).
- **حالت اقتصادی:** پردازه‌ها را **یکی‌یکی** بکشیم تا وقتی که بن‌بست رفع شود.
  - *مزیت:* تلفات کمتر است.
  - *عیب:* بعد از کشتن هر پردازه، باید دوباره الگوریتم تشخیص را اجرا کنیم ببینیم درست شد یا نه (سربار بالا).

چه کسی را قربانی کنیم؟ (معیارهای انتخاب قربانی):

1. اولویت پردازه (کمترها قربانی می‌شوند).
2. زمان باقی‌مانده (آن‌هایی که کارشان رو به اتمام است را نکشیم بهتر است).
3. حجم منابعی که گرفته (هر که منابع بیشتری اشغال کرده، کشتنش سودمندتر است!).

روش ۲: پس‌گیری منابع (Resource Preemption) 🔄

به جای کشتن پردازه، فقط منابعش را به زور می‌گیریم و به دیگری می‌دهیم.

در این روش باید به ۳ سوال پاسخ دهیم:

1. **انتخاب قربانی (Selecting a Victim):** کدام نخ را لخت کنیم؟ (معمولاً بر اساس هزینه).
2. **بازگشت به عقب (Rollback):** وقتی منبع را گرفتیم، با نخ چه کنیم؟
   - نمی‌تواند ادامه دهد (چون منبع ندارد).
   - باید آن را به یک **حالت امن** در گذشته برگردانیم (Rollback) و دوباره از آنجا اجرا کنیم. (ساده‌ترین حالت: ریستارت کامل پردازه).

قحطی‌زدگی (Starvation): نکند همیشه یک نخ بدبخت را قربانی کنیم؟

- *راه‌حل:* تعداد دفعاتی که یک نخ قربانی شده را بشماریم و در انتخاب بعدی لحاظ کنیم (کسی که زیاد قربانی شده، مصونیت بگیرد).

🌟 جدول جمع‌بندی نهایی فصل ۸ (کلیدواژه‌های امتحانی)

مبحث	خلاصه نکته
Wait-for Graph	فقط برای منابع **تک‌نمونه‌ای**. دور = بن‌بست.
تشخیص چندنمونه‌ای	شبیه بانکدار ولی با ماتریس Request. اگر Finish فالس ماند = بن‌بست.
ترمیم با کشتن	یا همه را می‌کشیم یا یکی‌یکی (نیاز به اجرای مجدد تشخیص).
ترمیم با پس‌گیری	چالش‌ها: انتخاب قربانی، Rollback، جلوگیری از قحطی.
الگوریتم شترمرغ	(از بخش‌های قبل) نادیده گرفتن کل ماجرا! (روش ویندوز/لینوکس).